PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Thanush	**Roll No:** 26000176
Branch: CSE	**Course:** Python Programming

Case 1: Progressive Electricity Billing

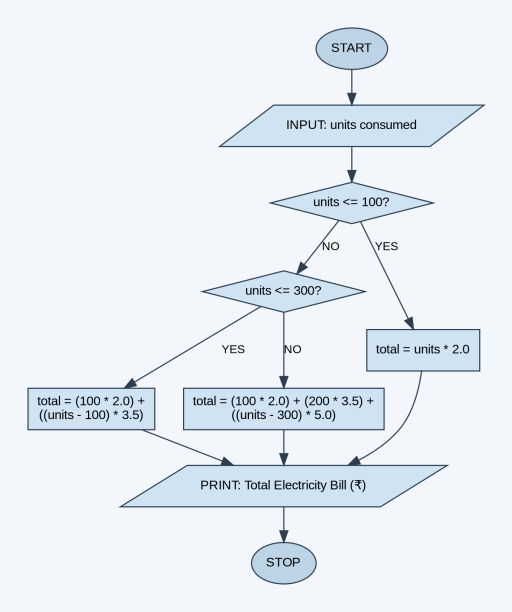
Problem Statement

Write a Python program to calculate the progressive electricity bill based on units consumed using tiered pricing rules (Rs. 2.0 per unit for the first 100 units, Rs. 3.5 for the next 200 units, and Rs. 5.0 for every unit above 300).

Algorithm

- Step 1: Start.
- Step 2: Read units consumed from user.
- Step 3: If units ≤ 100, total = units × 2.0.
- Step 4: Else if units ≤ 300, total = (100 × 2.0) + ((units - 100) × 3.5).
- Step 5: Else total = (100 × 2.0) + (200 × 3.5) + ((units - 300) × 5.0).
- Step 6: Print the total electricity bill (rounded to 2 decimals). Stop.

Flowchart



Source Code & Output

```
units = float(input("Enter units consumed: "))

if units <= 100:
    total = units * 2.0
elif units <= 300:
    total = (100 * 2.0) + ((units - 100) * 3.5)
else:
    total = (100 * 2.0) + (200 * 3.5) + ((units - 300) * 5.0)

print("Total Electricity Bill: ₹", round(total, 2))
```

```
Enter units consumed: 100 | Total Electricity Bill: ₹ 200.0
```

Case 2: ATM Transaction Simulation

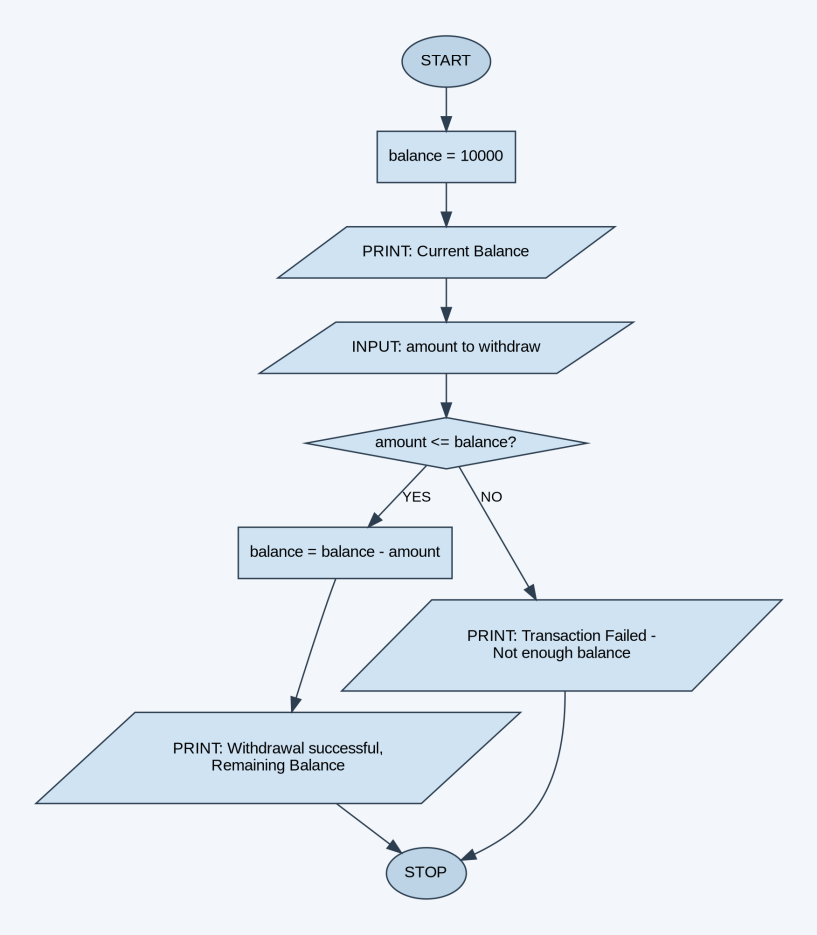
Problem Statement

Simulate an ATM withdrawal with an opening balance of Rs. 10000. The user enters the amount to withdraw; the withdrawal succeeds only if the amount does not exceed the available balance, otherwise the transaction fails.

Algorithm

- Step 1: Start.
- Step 2: Set balance = 10000 and display the current balance.
- Step 3: Read the amount to withdraw from user.
- Step 4: If amount ≤ balance, balance = balance - amount and print the remaining balance.
- Step 5: Otherwise print that the transaction failed due to insufficient balance.
- Step 6: Stop.

Flowchart



Source Code & Output

```
balance = 10000
print("Current Balance: ₹", balance)

amount = float(input("Enter amount to withdraw: "))

if amount <= balance:
    balance = balance - amount
    print("Withdrawal successful! Remaining Balance: ₹", balance)
else:
    print("Transaction Failed: Not enough balance!")
```

```
Current Balance: ₹ 10000 | Enter amount to withdraw: 1000
Withdrawal successful! Remaining Balance: ₹ 9000.0
```

Case 3: Attendance Recovery Computation

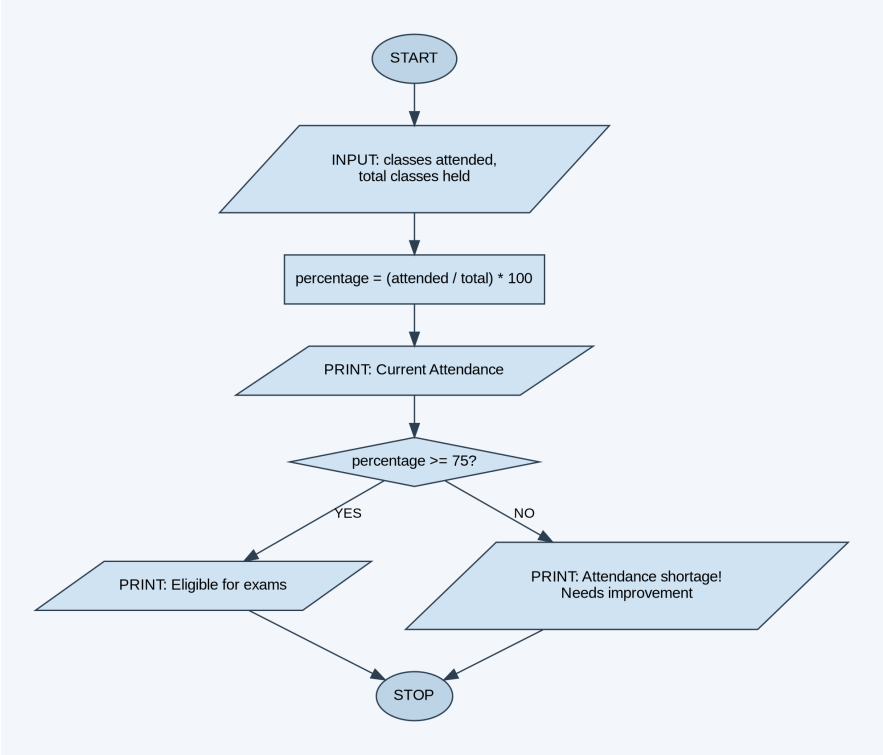
Problem Statement

Calculate a student's current attendance percentage from the classes attended and the total classes held, and check whether the student is eligible for exams (minimum 75% attendance required).

Algorithm

- Step 1: Start.
- Step 2: Read classes attended and total classes held from user.
- Step 3: Calculate percentage = (attended / total) × 100.
- Step 4: Print the current attendance percentage.
- Step 5: If percentage ≥ 75, print "Eligible for exams."
- Step 6: Otherwise print "Attendance shortage! Needs improvement." Stop.

Flowchart



Source Code & Output

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes held: "))

percentage = (attended / total) * 100
print(f"Current Attendance: {percentage:.2f}%")

if percentage >= 75:
    print("Eligible for exams.")
else:
    print("Attendance shortage! Needs improvement.")
```

```
Enter classes attended: 75 | Enter total classes held: 100 | Current Attendance: 75.00%
Eligible for exams.
```

Case 4: Password Policy Validation Engine

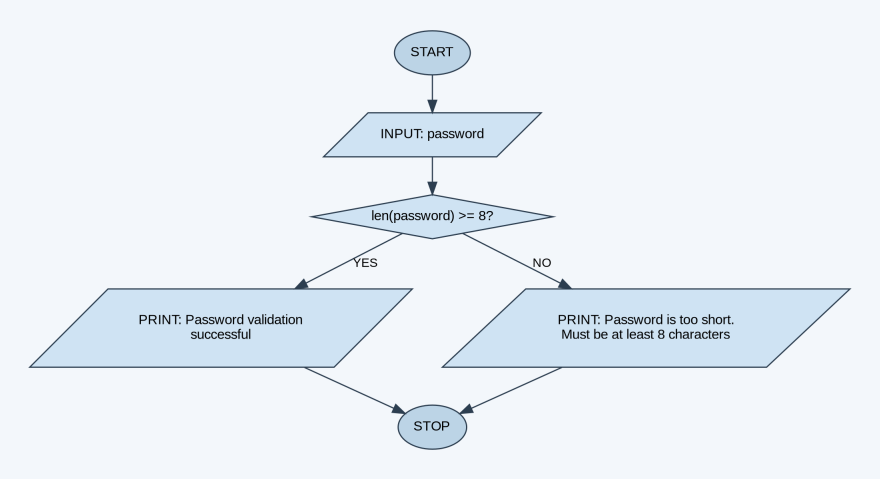
Problem Statement

Evaluate a password string against a minimum-length policy. The password is valid only if it contains at least 8 characters; otherwise an error message is shown.

Algorithm

- Step 1: Start.
- Step 2: Read the password from user.
- Step 3: Find the length of the password using len().
- Step 4: If length ≥ 8, print "Password validation successful."
- Step 5: Otherwise print "Password is too short. Must be at least 8 characters."
- Step 6: Stop.

Flowchart



Source Code & Output

```
password = input("Enter your password: ")

if len(password) >= 8:
    print("Password validation successful.")
else:
    print("Password is too short. Must be at least 8 characters.")
```

```
Enter your password: 12345 | Password is too short. Must be at least 8 characters.
```

Case 5: Multi-Product Shopping Budget Constraint

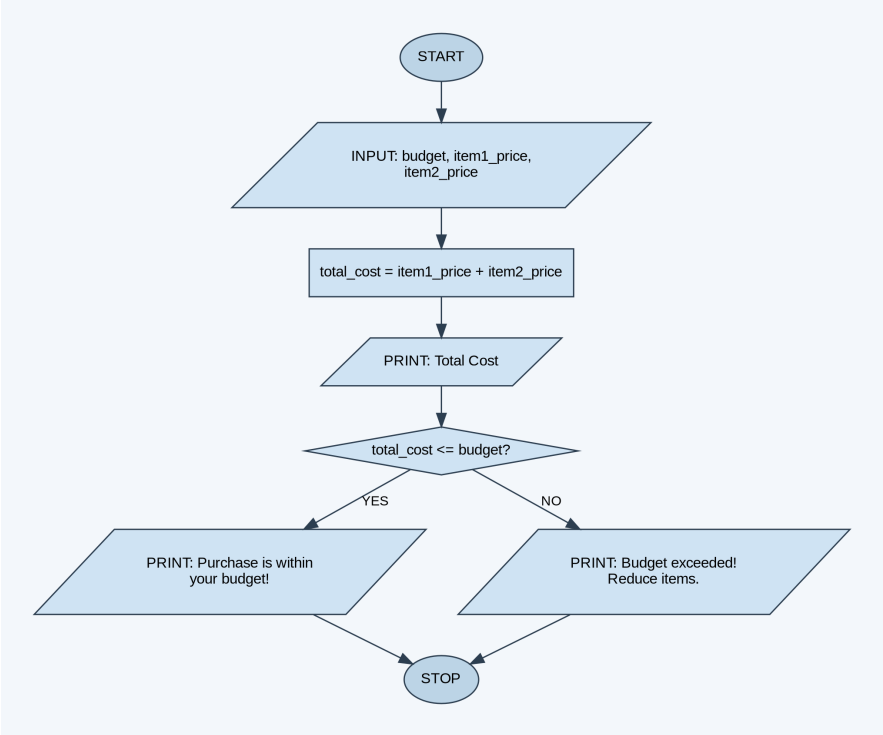
Problem Statement

Evaluate a shopping basket containing two items against a fixed budget. Calculate the total cost of the items and check whether the purchase is within the budget.

Algorithm

- Step 1: Start.
- Step 2: Read the total budget, price of item 1 and price of item 2 from user.
- Step 3: Calculate total_cost = item1_price + item2_price.
- Step 4: Print the total cost.
- Step 5: If total_cost ≤ budget, print "Purchase is within your budget!"
- Step 6: Otherwise print "Budget exceeded! Reduce items." Stop.

Flowchart



Source Code & Output

```
budget = float(input("Enter total budget: "))
item1_price = float(input("Enter price of item 1: "))
item2_price = float(input("Enter price of item 2: "))

total_cost = item1_price + item2_price
print("Total Cost:", total_cost)

if total_cost <= budget:
    print("Purchase is within your budget!")
else:
    print("Budget exceeded! Reduce items.")
```

```
Budget: 5000 | Item 1: 2500 | Item 2: 2400 | Total Cost: 4900.0
Purchase is within your budget!
```

Case 6: Employee Payroll and Deduction Analysis

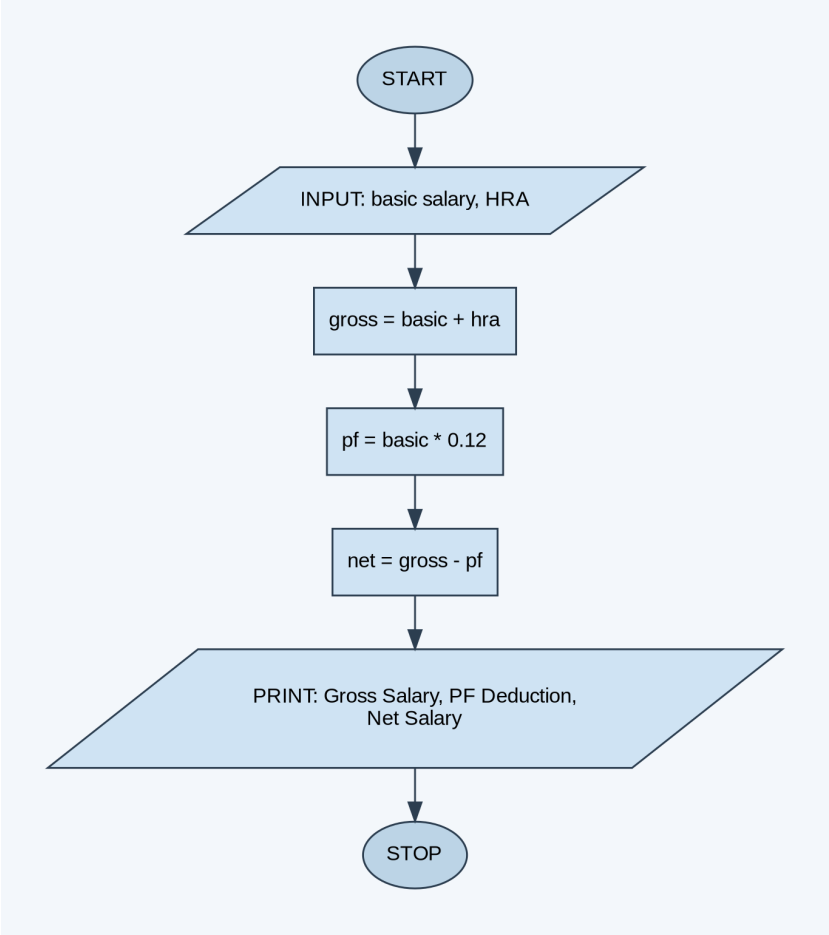
Problem Statement

Calculate the gross salary, Provident Fund (PF) deduction and net salary of an employee. Gross salary is the sum of basic salary and HRA, and PF is deducted at 12% of the basic salary.

Algorithm

- Step 1: Start.
- Step 2: Read basic salary and HRA from user.
- Step 3: Calculate gross = basic + hra.
- Step 4: Calculate pf = basic × 0.12.
- Step 5: Calculate net = gross - pf.
- Step 6: Print gross salary, PF deduction and net salary. Stop.

Flowchart



Source Code & Output

```
basic = float(input("Enter Basic Salary: "))
hra = float(input("Enter HRA: "))

gross = basic + hra
pf = basic * 0.12
net = gross - pf

print("Gross Salary: ₹", gross)
print("PF Deduction: ₹", pf)
print("Net Salary: ₹", net)
```

```
Basic Salary: 10000 | HRA: 2000 | Gross Salary: ₹ 12000.0 | PF Deduction: ₹ 1200.0
Net Salary: ₹ 10800.0
```

Case 7: Student Marks and Result Evaluation

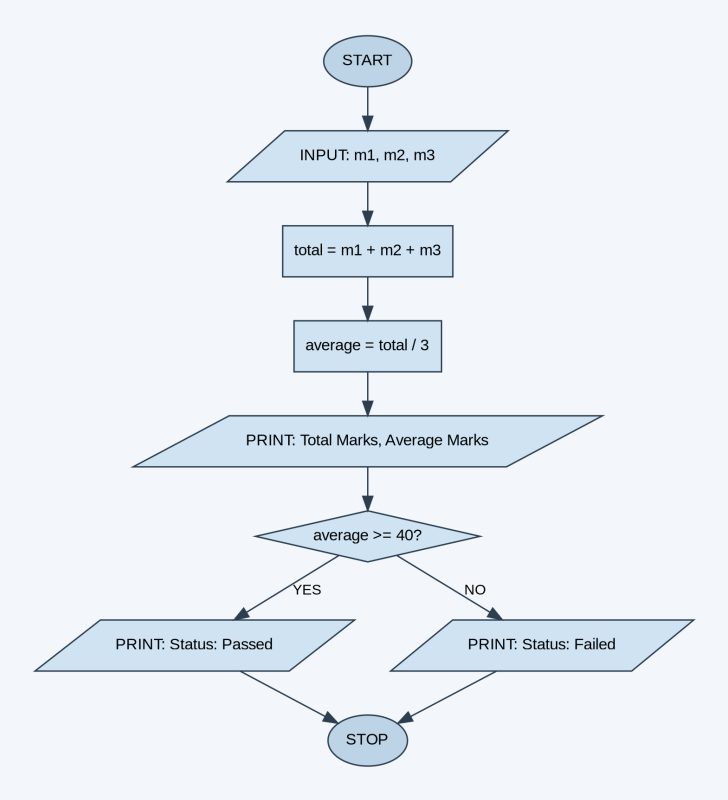
Problem Statement

Read the marks of a student in 3 subjects, calculate the total and average marks, and declare the result as Passed if the average is at least 40, otherwise Failed.

Algorithm

- Step 1: Start.
- Step 2: Read the marks of subject 1, subject 2 and subject 3 from user.
- Step 3: Calculate total = m1 + m2 + m3.
- Step 4: Calculate average = total / 3.
- Step 5: Print the total marks and the average marks (rounded to 2 decimals).
- Step 6: If average ≥ 40, print "Status: Passed"; otherwise print "Status: Failed". Stop.

Flowchart



Source Code & Output

```
print("Enter marks for 3 subjects:")
m1 = float(input("Subject 1 marks: "))
m2 = float(input("Subject 2 marks: "))
m3 = float(input("Subject 3 marks: "))
total = m1 + m2 + m3
average = total / 3
print("\n--- Results ---")
print("Total Marks:", total)
print("Average Marks:", round(average, 2))
if average >= 40:
    print("Status: Passed")
else:
    print("Status: Failed")
```

```
Subject 1: 20 | Subject 2: 20 | Subject 3: 20 | Total Marks: 60.0 | Average Marks: 20.0
Status: Failed
```

Case 8: Smart Canteen Order and Stock Management

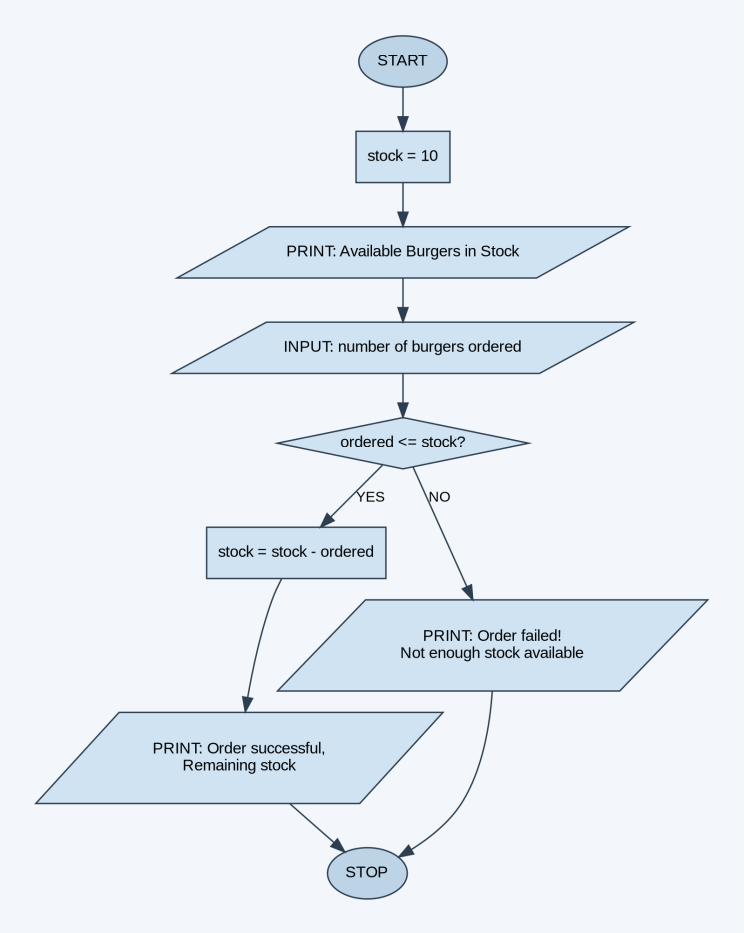
Problem Statement

Manage canteen burger orders against the available stock of 10 burgers. If the ordered quantity is within the stock, the order succeeds and the stock is reduced; otherwise the order fails.

Algorithm

- Step 1: Start.
- Step 2: Set stock = 10 and display the available burgers.
- Step 3: Read the number of burgers ordered from user.
- Step 4: If ordered ≤ stock, stock = stock - ordered; print "Order successful!" and the remaining stock.
- Step 5: Otherwise print "Order failed! Not enough stock available."
- Step 6: Stop.

Flowchart



Source Code & Output

```
stock = 10
print("Available Burgers in Stock:", stock)

ordered = int(input("Enter number of burgers you want to buy: "))

if ordered <= stock:
    stock = stock - ordered
    print("Order successful!")
    print("Remaining stock in canteen:", stock)
else:
    print("Order failed! Not enough stock available.")
```

```
Available Burgers in Stock: 10 | Burgers to buy: 20
Order failed! Not enough stock available.
```

Case 9: Library Borrowing and Fine Reconciliation

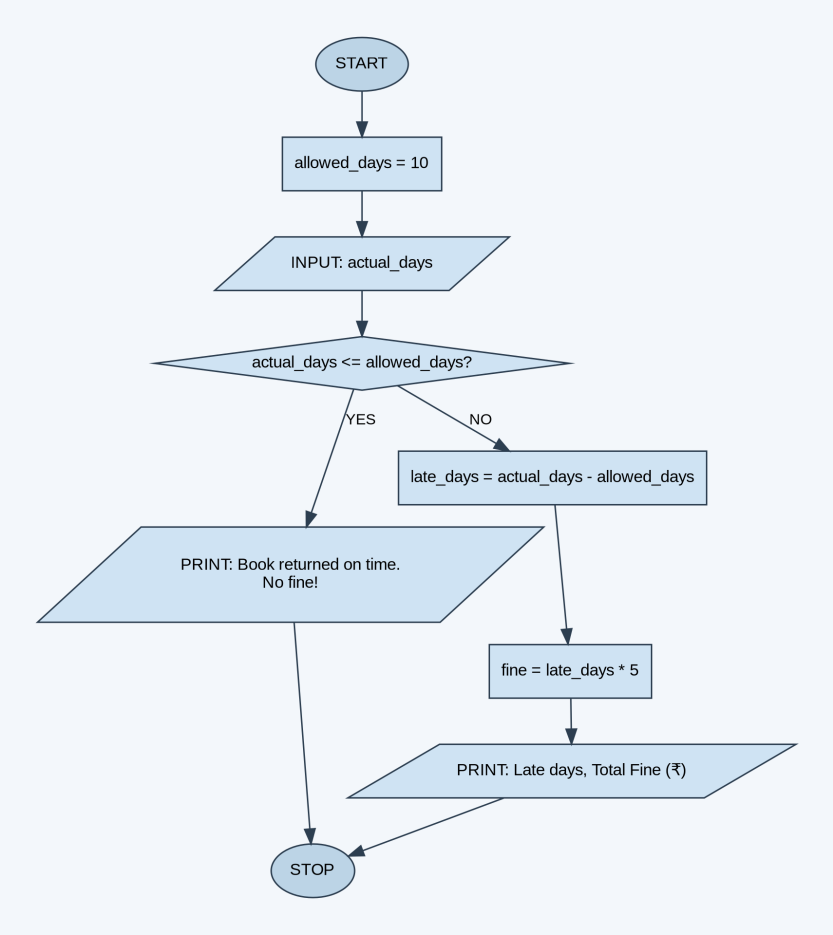
Problem Statement

Analyse a book borrowing period and calculate the library fine. A book may be kept for 10 days, and a fine of Rs. 5 per day is charged for every day beyond that.

Algorithm

- Step 1: Start.
- Step 2: Set allowed_days = 10.
- Step 3: Read the actual days taken to return the book.
- Step 4: If actual_days ≤ allowed_days, print that the book was returned on time with no fine.
- Step 5: Otherwise late_days = actual_days - allowed_days, fine = late_days × 5; print the late days and the total fine.
- Step 6: Stop.

Flowchart



Source Code & Output

```
allowed_days = 10
actual_days = int(input("Enter actual days taken to return the book: "))

if actual_days <= allowed_days:
    print("Book returned on time. No fine!")
else:
    late_days = actual_days - allowed_days
    fine = late_days * 5
    print("Book is late by", late_days, "days.")
    print("Total Fine to Pay: ₹", fine)
```

```
Enter actual days taken to return the book: 20 | Book is late by 10 days.
Total Fine to Pay: ₹ 50
```

Case 10: Check Even or Odd

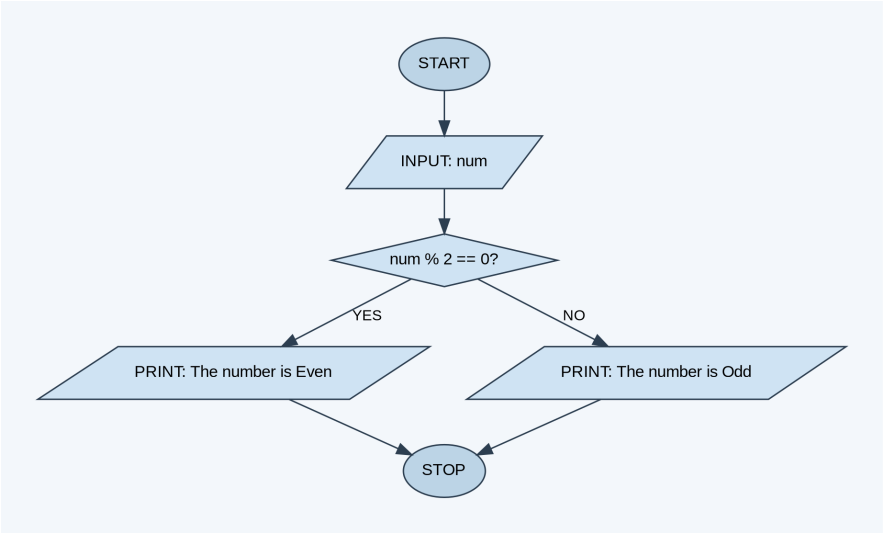
Problem Statement

Write a simple Python program to check whether a number entered by the user is even or odd.

Algorithm

- Step 1: Start.
- Step 2: Read a number from user.
- Step 3: If num % 2 == 0, print "The number is Even".
- Step 4: Otherwise print "The number is Odd".
- Step 5: Stop.

Flowchart



Source Code & Output

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("The number is Even")
else:
    print("The number is Odd")
```

```
Enter a number: 230 | The number is Even
```